

DBMS SKILL TASK – WEEK 6

Product Review and Rating Management System

Introduction

The Product Review and Rating Management System is developed as part of the Nykaa DBMS project to manage customer feedback and product ratings efficiently. The system stores reviews provided by customers along with their ratings for different products. It helps retrieve product review details, calculate the average rating of products using aggregate functions, identify highly rated products, and generate a product rating report. This system provides a structured way to manage customer feedback and supports better understanding of product performance based on customer ratings.

OBJECTIVE

- To create a Review table for storing customer product reviews.
- To create a Rating table for storing product ratings.
- To store customer feedback and ratings for Nykaa products.
- To retrieve product review details using SQL queries.
- To calculate average product ratings using aggregate functions.
- To identify highly rated products based on their average ratings.
- To generate a Product Rating Report using the Product and Rating tables.

1. Database design

1.1 Review table

Attribute	Data Type	Constraint
Review_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key
Product_ID	NUMBER	Foreign Key
Review_Text	VARCHAR2(500)	NOT NULL
Review_Date	DATE	NOT NULL

1.2 Rating Table

Attribute	Data Type	Constraint
Rating_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key
Product_ID	NUMBER	Foreign Key
Rating	NUMBER(1)	NOT NULL, 1–5
Rating_Date	DATE	NOT NULL

2. TABLE RELATIONSHIPS

- **Customer → Review**
One customer can write many reviews.
Customer.Customer_ID → Review.Customer_ID
Relationship: 1 : M
- **Product → Review**
One product can receive many reviews.
Product.Product_ID → Review.Product_ID
Relationship: 1 : M
- **Customer → Rating**
One customer can give ratings to multiple products.
Customer.Customer_ID → Rating.Customer_ID
Relationship: 1 : M
- **Product → Rating**
One product can receive ratings from multiple customers.
Product.Product_ID → Rating.Product_ID
Relationship: 1 : M

3. Table creation

3.1 Create Review table

```
SQL> CREATE TABLE Review (  
 2     Review_ID NUMBER PRIMARY KEY,  
 3     Customer_ID NUMBER,  
 4     Product_ID NUMBER,  
 5     Review_Text VARCHAR2(500) NOT NULL,  
 6     Review_Date DATE NOT NULL,  
 7     CONSTRAINT fk_review_customer  
 8         FOREIGN KEY (Customer_ID)  
 9         REFERENCES Customer(Customer_ID),  
10     CONSTRAINT fk_review_product  
11         FOREIGN KEY (Product_ID)  
12         REFERENCES Product(Product_ID)  
13 );
```

3.2 Rating table

```
SQL> CREATE TABLE Rating (  
 2     Rating_ID NUMBER PRIMARY KEY,  
 3     Customer_ID NUMBER,  
 4     Product_ID NUMBER,  
 5     Rating NUMBER(1) NOT NULL,  
 6     Rating_Date DATE NOT NULL,  
 7     CONSTRAINT fk_rating_customer  
 8         FOREIGN KEY (Customer_ID)  
 9         REFERENCES Customer(Customer_ID),  
10     CONSTRAINT fk_rating_product  
11         FOREIGN KEY (Product_ID)  
12         REFERENCES Product(Product_ID),  
13     CONSTRAINT chk_rating  
14         CHECK (Rating BETWEEN 1 AND 5)  
15 );
```

Table created.

4. Review Data insertion

```
SQL> INSERT INTO Review
  2 VALUES (1, 101, 1001, 'Very good face serum and makes my skin feel fresh.', DATE '2026-10-01');
1 row created.

SQL>
SQL> INSERT INTO Review
  2 VALUES (2, 102, 1002, 'The lipstick has a nice matte finish and good colour.', DATE '2026-10-02');
1 row created.

SQL>
SQL> INSERT INTO Review
  2 VALUES (3, 103, 1003, 'Good shampoo and it helps reduce dandruff.', DATE '2026-10-03');
1 row created.

SQL>
SQL> INSERT INTO Review
  2 VALUES (4, 104, 1004, 'The perfume has a pleasant fragrance and lasts long.', DATE '2026-10-04');
1 row created.

SQL>
SQL> INSERT INTO Review
  2 VALUES (5, 105, 1005, 'The body lotion is smooth and moisturizing.', DATE '2026-10-05');
1 row created.
```

5. Rating data insertion

```
SQL> INSERT INTO Rating
  2 VALUES (1, 101, 1001, 5, DATE '2026-10-01');
1 row created.

SQL>
SQL> INSERT INTO Rating
  2 VALUES (2, 102, 1002, 4, DATE '2026-10-02');
1 row created.

SQL>
SQL> INSERT INTO Rating
  2 VALUES (3, 103, 1003, 5, DATE '2026-10-03');
1 row created.

SQL>
SQL> INSERT INTO Rating
  2 VALUES (4, 104, 1004, 4, DATE '2026-10-04');
1 row created.

SQL>
SQL> INSERT INTO Rating
  2 VALUES (5, 105, 1005, 5, DATE '2026-10-05');
1 row created.
```

6. Retrieve product review details

```
SQL> SELECT
  2   r.Review_ID,
  3   r.Customer_ID,
  4   r.Product_ID,
  5   p.Product_Name,
  6   r.Review_Text,
  7   r.Review_Date
  8 FROM Review r
  9 JOIN Product p
10 ON r.Product_ID = p.Product_ID
11 ORDER BY r.Review_Date;

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
      1          101          1001
Vitamin C Face Serum
Very good face serum and makes my skin feel fresh.
01-OCT-26

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
      2          102          1002
Matte Lipstick
The lipstick has a nice matte finish and good colour.
02-OCT-26

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
      3          101          1003
Anti Dandruff Shampoo
Good shampoo and it helps reduce dandruff.
01-OCT-26

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
      4          104          1004
Floral Eau De Parfum
The perfume has a pleasant fragrance and lasts long.
04-OCT-26

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
      5          105          1005
Body Lotion
The body lotion is smooth and moisturizing.
05-OCT-26
```

7. Calculate average product ratings

```
SQL> SELECT
  2   Product_ID,
  3   ROUND(AVG(Rating), 2) AS Average_Rating
  4 FROM Rating
  5 GROUP BY Product_ID
  6 ORDER BY Average_Rating DESC;

PRODUCT_ID AVERAGE_RATING
-----
1001          5
1003          5
1005          5
1004          4
1002          4
```

8. Identify highly rated products

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     ROUND(AVG(r.Rating), 2) AS Average_Rating
  5 FROM Product p
  6 JOIN Rating r
  7 ON p.Product_ID = r.Product_ID
  8 GROUP BY p.Product_ID, p.Product_Name
  9 HAVING AVG(r.Rating) >= 4
 10 ORDER BY Average_Rating DESC;
```

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

1001
Vitamin C Face Serum
5

1003
Anti Dandruff Shampoo
5

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

1005
Body Lotion
5

1004
Floral Eau De Parfum

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

4

1002
Matte Lipstick
4

9. Product rating report

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     COUNT(r.Rating_ID) AS Total_Ratings,
  5     ROUND(AVG(r.Rating), 2) AS Average_Rating,
  6     MIN(r.Rating) AS Minimum_Rating,
  7     MAX(r.Rating) AS Maximum_Rating
  8 FROM Product p
  9 JOIN Rating r
10 ON p.Product_ID = r.Product_ID
11 GROUP BY p.Product_ID, p.Product_Name
12 ORDER BY Average_Rating DESC;
```

PRODUCT_ID

PRODUCT_NAME

TOTAL_RATINGS	AVERAGE_RATING	MINIMUM_RATING	MAXIMUM_RATING
---------------	----------------	----------------	----------------

1001			
Vitamin C Face Serum			
1	5	5	5

1003			
Anti Dandruff Shampoo			
1	5	5	5

PRODUCT_ID

PRODUCT_NAME

TOTAL_RATINGS	AVERAGE_RATING	MINIMUM_RATING	MAXIMUM_RATING
---------------	----------------	----------------	----------------

1005			
Body Lotion			
1	5	5	5

1004			
Floral Eau De Parfum			

PRODUCT_ID

PRODUCT_NAME

TOTAL_RATINGS	AVERAGE_RATING	MINIMUM_RATING	MAXIMUM_RATING
---------------	----------------	----------------	----------------

1	4	4	4
---	---	---	---

1002			
Matte Lipstick			
1	4	4	4